



ПРОЕКТ
МЭРА
МОСКВЫ

БИЗНЕС
МОСКВЫ



РАЗВИТИЕ
ЧЕЛОВЕЧЕСКОГО
КАПИТАЛА



ЛИДЕРЫ
ЦИФРОВОЙ
ТРАНСФОРМАЦИИ

Полуночники

Москва 02: Сервис моделирования трассировки тепловых сетей для технологического присоединения новых зданий

КОМАНДА «ПОЛУНОЧНИКИ»

Краткое описание решения:

Система автоматического проектирования вариантов подключения объектов к существующей теплосети в 2D и 3D с учётом стоимости, инженерных ограничений, препятствий и параметров сети.

Уникальность решения:

Вместо независимой прокладки трассы к каждому объекту решение строит **общую разветвлённую сеть с переиспользованием магистральных участков, использует точный sparse visibility graph по геометрии препятствий без растривания территории** и затем совместно оптимизирует положения камер и точек врезки с помощью ДП по дереву сети. Дополнительно алгоритм умеет автоматически переподключать уже построенные ветви к более выгодной топологии и формирует несколько различающихся инженерно валидных вариантов.

О команде

- **Капитан:** Марчук Софья
- **Кол-во участников:** 4 человека
- **Краткое описание:**
 - познакомились на олимпиадах и проектных сменах
 - участвуем в хакатонах с 2023 года
- **Город и регион:** Москва и Санкт-Петербург

КОМАНДА «ПОЛУНОЧНИКИ»



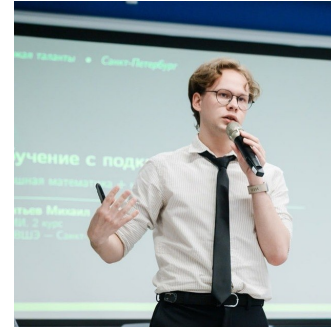
**Софья
Марчук**

- Разработчик алгоритма
- @scarlettnik
- +79850436933
- МФТИ



**Андрей
Кругликов**

- Java backend dev
- @KruASe
- +79037711768
- Т-Банк, ВШПИ МФТИ



**Михаил
Леонтьев**

- Геоаналитика
- @michlea_tg
- +79297114308
- ВШЭ



**Кирилл
Черников**

- Муза
- @data_unsorted
- +79198072006
- ИТМО

Краткая история команды:

Знакомились, оказываясь вместе на различных олимпиадах и проектных сменах, таких как «НТО» и «Большие вызовы»

Участвуем в хакатонах с 2023 года

01

Почему вы выбрали именно эту задачу из предложенных на хакатоне?

Не хотим, чтобы зимой было холодно

02

С какими основными сложностями или вызовами вы столкнулись и как их преодолели?

Самой сложной частью оказалось заставить алгоритм правильно выбирать сторону, с которой труба должна заходить в здание

03

Техническая суть решения

- Оптимизируем не отдельные маршруты, а всю сеть целиком
- Используем точную геометрию вместо растрования территории
- Маршрутизация сразу учитывает диаметр трубы
- Поддерживаем инженерно корректный вход в здания
- Автоматически рассчитываем расход и DN по всей топологии
- Совместно оптимизируем положение камер и геометрию ветвей
- Алгоритм умеет пересматривать уже принятые решения
- 2D и 3D являются частью одной модели
- Каждый вариант проходит независимую инженерную валидацию
- Решение dataset-agnostic

Маркетинговая суть решения

- Проект автоматизирует инженерное проектирование, а не просто рисование маршрута
- Экономия достигается на уровне всей сети
- Решение работает со сложной реальной городской геометрией
- Система предлагает несколько качественно разных вариантов
- Проект снижает объём ручной работы инженера
- Алгоритм способен улучшать решение после его первоначального построения
- Архитектура рассчитана на промышленное использование
- Результат можно сразу использовать в существующих GIS-процессах
- Главное конкурентное отличие — сочетание оптимизации, инженерных правил и точной геометрии в одном цикле

ТЕПЛОКОНТУР

Сервис автоматического моделирования трасс подключения к тепловым сетям

Финальная техническая презентация · Java 11 · Spring Boot · потоковый GeoJSON · 2D + Depth routing

17/17

ОКС подключены

13.8848

лучший score

273.2 млн

стоимость варианта

1.86 км

линейные работы

validation_passed = true

status DONE

Scenario 8 · 144 объектов



Финальный прогон: расчёт завершён и валидирован

Данные из локального RUN_TASK: 2D и 3D дошли до DONE, все варианты подключают 17/17 ОКС

100%



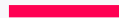
progress в 2D и 3D

0



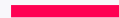
ошибок валидатора

3 + 3



варианта в выдаче

1.47%



улучшение к 13.4219

Что это значит

Сервис не просто сохранил старый результат из папки target: в новой выгрузке result-2d/result-3d содержатся три валидных варианта, рассчитанных через API-run. Варианты ранжированы по официальной формуле 70/30, пересчитанной без расхождений.

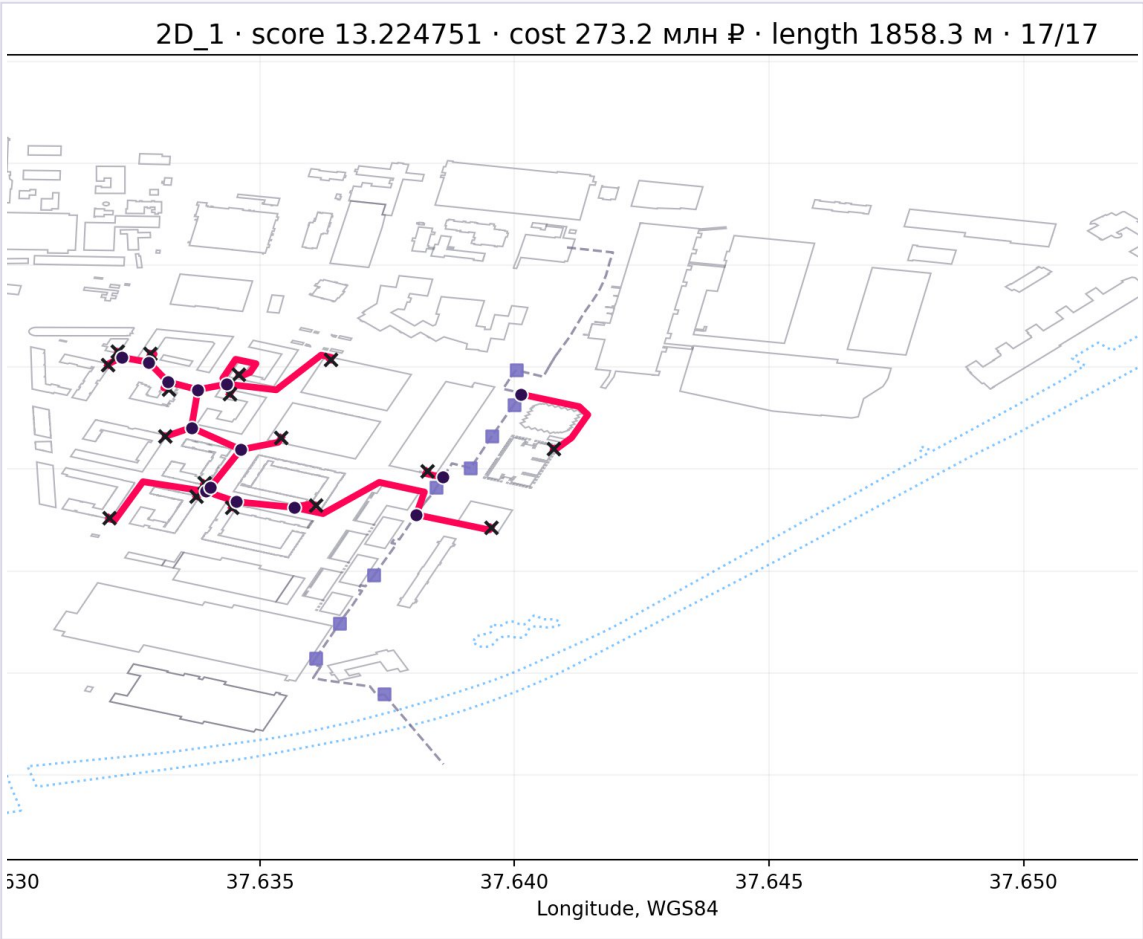
Ключевой результат

Лучший вариант — INFRASTRUCTURE_LIGHT: score 13.2247506, стоимость 273.214 млн Р, длина 1858.254 м. Вариант 2 дешевле по CAPEX, но длиннее; официальный score выбирает баланс стоимости и длины.

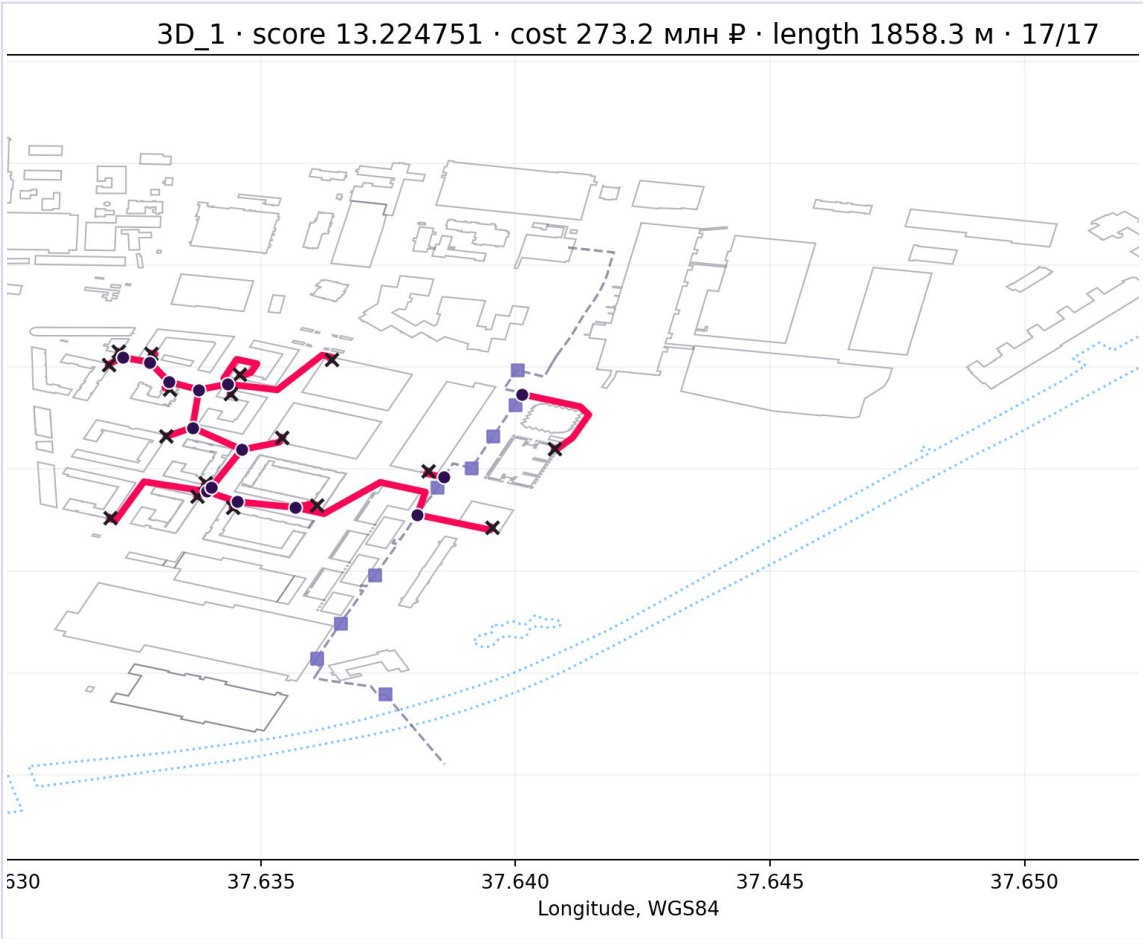
режим	rank	policy	score	стоимость	длина
2D	1	INFRASTRUCTURE_LIGHT	13.224751	273.2 млн	1858.3 м
2D	2	OFFICIAL_BEST	13.232146	268.8 млн	1902.2 м
2D	3	OFFICIAL_BEST	13.324118	276.3 млн	1862.9 м
3D	1	INFRASTRUCTURE_LIGHT	13.224751	273.2 млн	1858.3 м
3D	2	OFFICIAL_BEST	13.232146	268.8 млн	1902.2 м
3D	3	OFFICIAL_BEST	13.324118	276.3 млн	1862.9 м

Как выглядят маршруты: лучший 2D и лучший Depth

Розовая линия — новая сеть; серым показаны контуры ОКС/домов, фиолетовым — существующая теплосеть и камеры



2D №1 · score 13.2247506

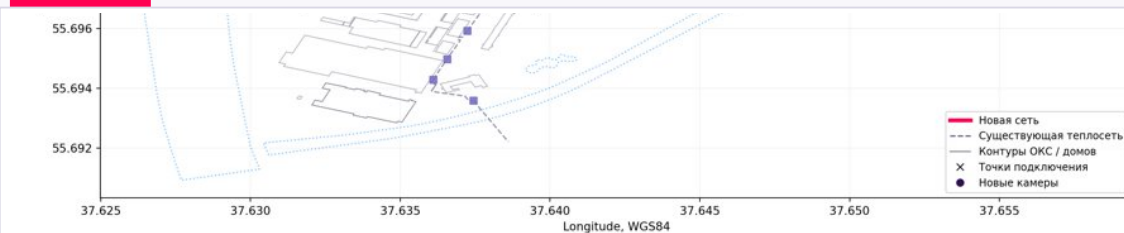


Depth №1 · score 13.2247506

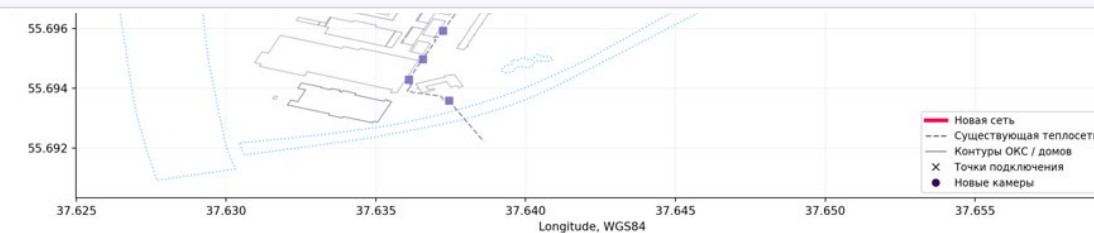
На данном датасете depth-run сохраняет ту же топологию и стоимость: это означает, что выбранная геометрия проходит правила глубины без ухудшения score.

Шесть вариантов для ручной проверки

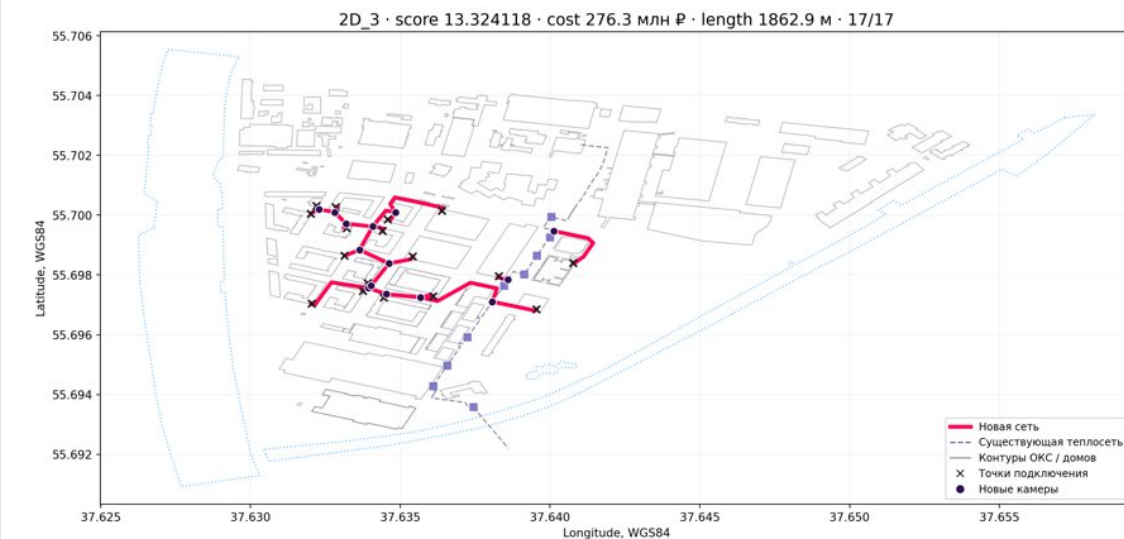
Содержательно разные трассы: INFRASTRUCTURE_LIGHT и OFFICIAL_BEST дают разные компромиссы стоимости и длины



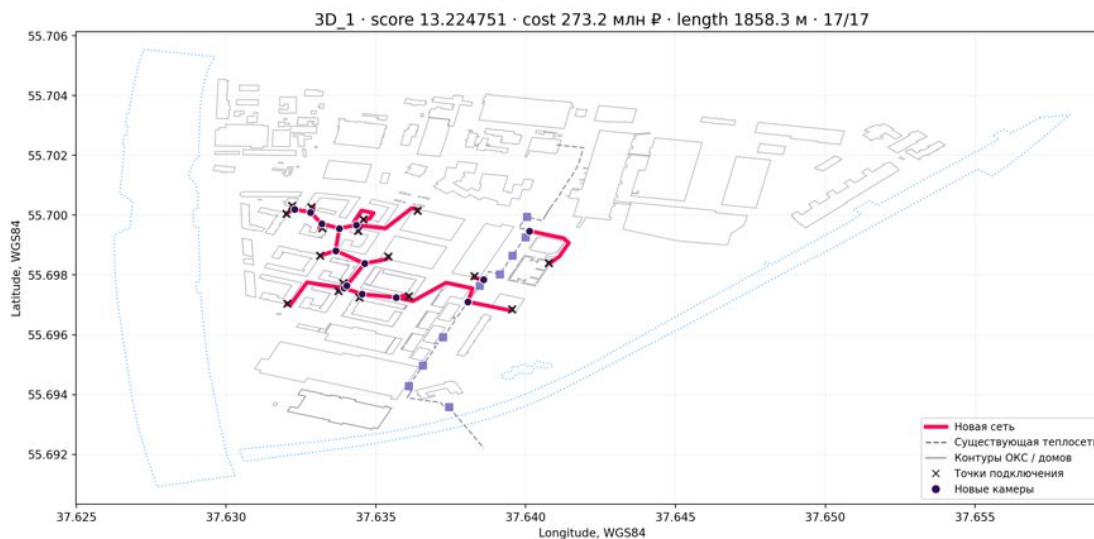
2D_3 · final user run



3D_1 · final user run



3D_2 · final user run



3D_3 · final user run



Все шесть GeoJSON сохранены отдельно: routes/2d_1.geojson ... routes/3d_3.geojson.

Задача не сводится к shortest path

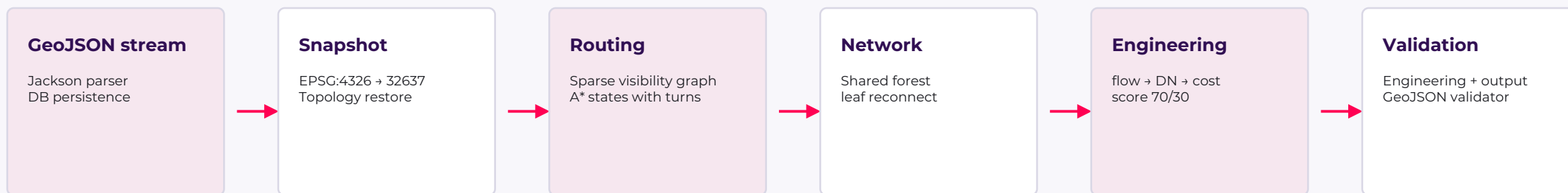
Каждый локальный выбор меняет стоимость всей конфигурации сети



Главный технический тезис: оптимизируется не линия, а целое дерево подключения с инженерным пересчётом после каждой структурной операции.

End-to-end pipeline решения

От потоковой загрузки GeoJSON до официальной выгрузки с тремя вариантами



Почему это важно

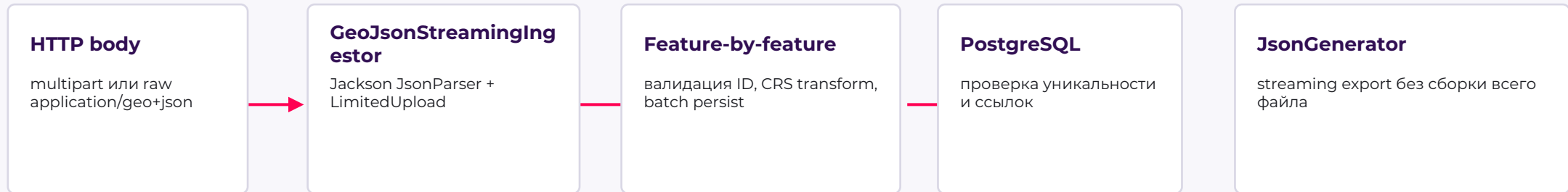
Поиск может использовать эвристики, но финальная выдача проходит строгую модель: суммы стоимости, геометрия, размерность координат, Z/depth, variant_summary и официальная формула score проверяются повторно.

Что получает эксперт

result-2d.geojson и result-3d.geojson — объединённые FeatureCollection с новыми участками, камерами и summary по каждому варианту; plus variants-*.json с PASS/ошибками.

Потоковая загрузка и выгрузка: теперь закрыто production-требование

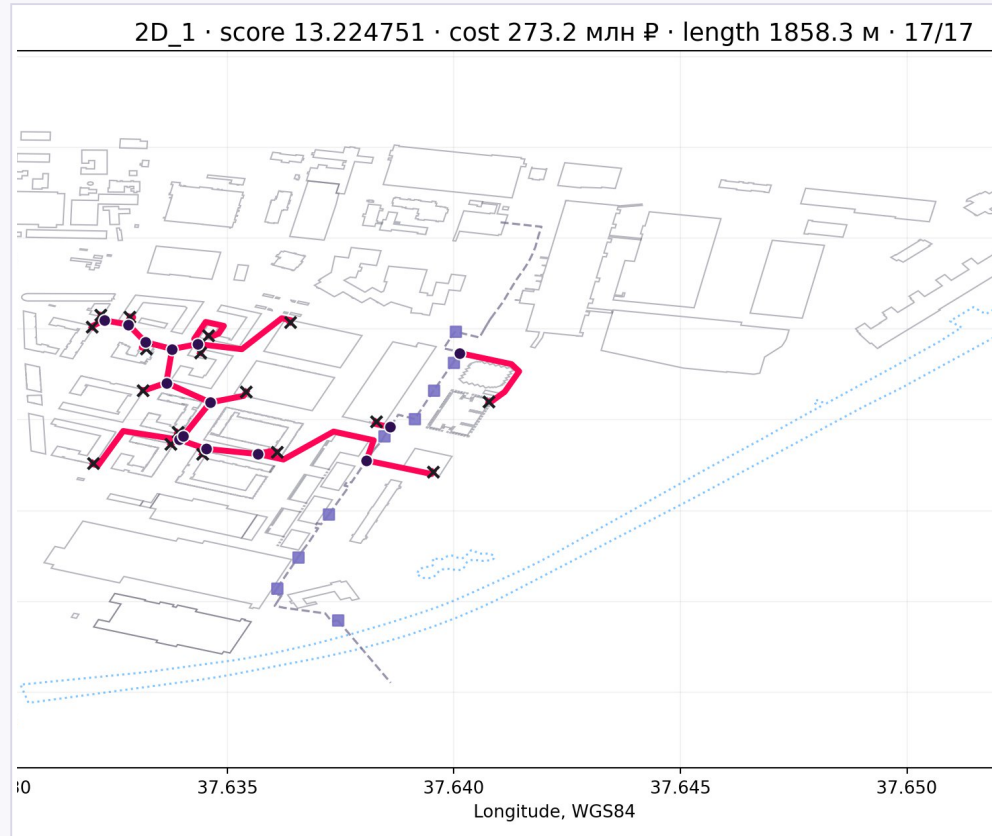
Файл до 3 ГБ не должен целиком попадать в память Java



- Сценарий из пользовательского прогона: READY, 144 features, diagnostic = null.
- 2D и Depth runs: status DONE, progress 100, error = null.
- Путь данных: InputStream → parser → объект → UTM geometry → DB; нет копии всего GeoJSON в памяти.
- Export использует JsonGenerator и формирует FeatureCollection потоково.

Геометрический слой: почему трасса не должна «залезать» на дома

Запрет транзита под зданиями встроен в RouteConstraintEngine, а не проверяется глазами после факта



Правило входа в ОКС

Полигон здания считается непроходимым. Единственное исключение — финальный прямой вход к своей точке подключения через ближайшую внешнюю стену. Вход через дальнюю стену, косой вход и транзит через другой корпус отклоняются.

DN-aware clearance

Для существующих ОКС действует горизонтальный отступ 5/7/9 м в зависимости от DN. Router строит граф на буферизованных препятствиях и проверяет legal edge перед добавлением ребра.

Почему визуально может казаться близко

Скриншот в WGS84 показывает осевую линию. Нормативные расстояния проверяются в EPSG:32637, а линия к своему ОКС входит только в финальном разрешённом участке.

SparseVisibilityRouter: точная геометрия вместо тяжёлого растра

Финальный router строит разреженный visibility graph по препятствиям и DN-буферам

1. Узлы графа

Буферизованные углы ограничений, подходы к зданию, точки врезки и точки подключения.

2. Рёбра видимости

Секторный visibility sweep: проверяются только геометрически перспективные связи, а не вся полная клика.

3. Состояние поиска

A*/Dijkstra по состояниям (node, previousNode) — чтобы контролировать угол поворота и запрет разворота внутри search.

Оптимизации производительности

- graph template кешируется на scenario/mode/DN
- route cache для запросов без новых линий
- corridor padding расширяется поэтапно
- occupied new lines могут только запретить путь, но не перестраивают всю статическую часть графа

Почему не AdaptiveGridRouter

Grid-подход удобен для раннего прототипа, но даёт лестничные/дискретные маршруты, дорожную сетку вокруг больших полигонов и сложнее контролирует «красивую» инженерную геометрию. Финальный путь — точные JTS-геометрии и разреженный visibility graph.

SharedNetworkBuilder: строим дерево подключений, а не набор независимых линий

Алгоритм выбирает, когда выгодно вести общую магистраль и где создавать новую камеру

Кандидаты подключения

Существующие камеры, проекции на существующие трубы, дискретные точки вдоль сети и присоединение к уже построенной новой ветви.

Последовательное построение леса

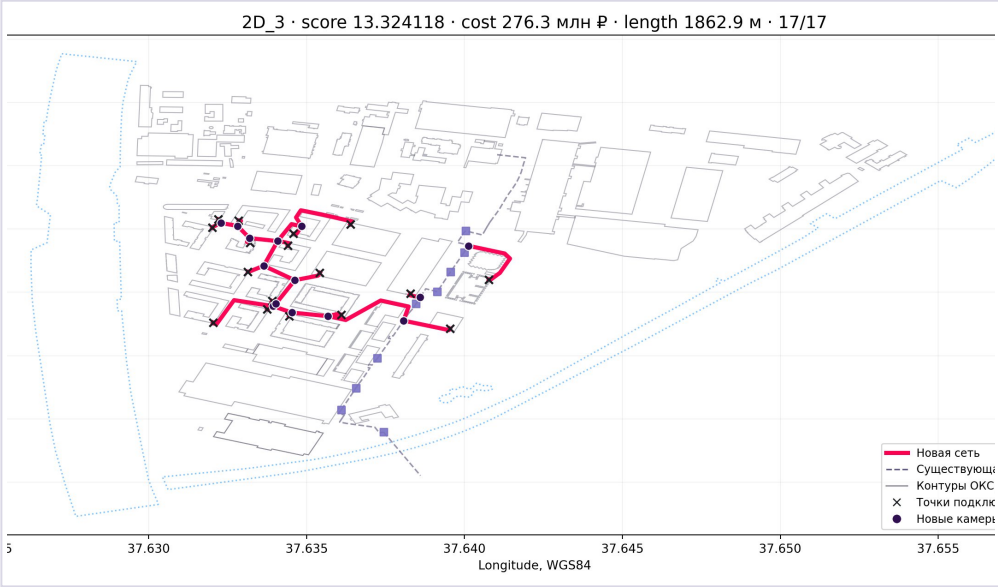
Для каждого ОКС проверяется подключение к существующей сети, новой камере или новой ветви. После подключения пересчитывается вся инженерная модель.

Reconnect leaves

После первичной топологии листья перекидываются к более выгодным узлам, затем заново запускается локальный refinement.

Финальный forest

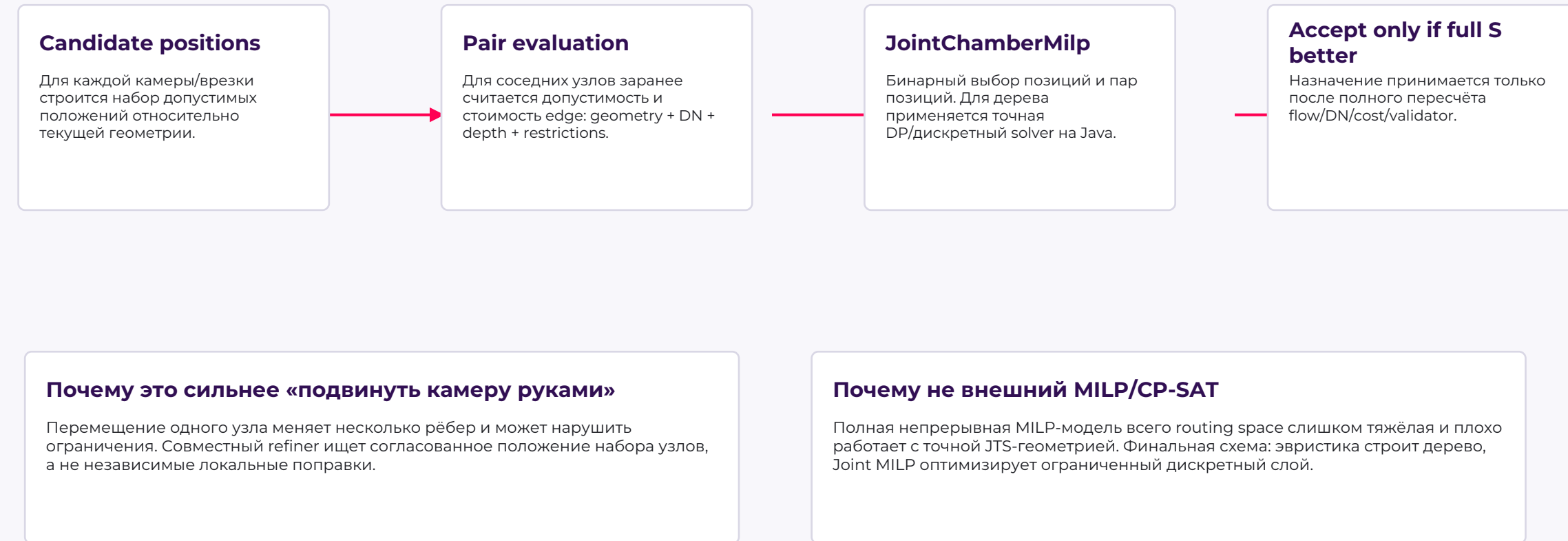
Не допускаются циклы, новые пересечения вне общего узла и камеры degree > 4; разные seed дают разные topology families.



Варианты 2D №2 и №3 показывают, что отбор возвращает не косметический сдвиг линии, а разные компромиссы топологии, камер и коридоров.

Совместная оптимизация камер и врезок

Локальная геометрия доводится не вручную, а дискретной оптимизационной моделью



Инженерная модель: flow → DN → length limit → cost

Каждый новый LineString имеет расчётный смысл, а не просто визуальную геометрию

Агрегация расходов

Если участок питает несколько ОКС, его flow_tph — сумма расходов downstream-потребителей. Подающий и обратный трубопровод не суммируются повторно.

DN selection

RuleBook.minForFlow выбирает минимальный DN с достаточной пропускной способностью. Same-DN max length проверяется ветвями.

Стоимость

Новые трубы: $L \times c(DN) \times Kdepth \times Kspecial$. Камеры — по шкале DN. Штраф неподключения = 0, так как подключено 17/17.

$$S = 0.7 \times C / 25\,000\,000 + 0.3 \times L / 100$$

для лучшего варианта: C = 273.214 млн Р, L = 1858.254 м, S = 13.2247506

Cost term



7.650

Length term



5.575

В этом запуске вариант №2 имеет меньшую стоимость, но проигрывает по длине; официальный score выбирает сбалансированный вариант №1.

Validation-first: search предлагает, validator решает

В презентацию и итоговую выдачу попадают только варианты, прошедшие независимую проверку

EngineeringValidator

degree ≤ 4 , no cycles, no new-new crossings, max turn, DN/flow/length, building entry, restrictions, depth feasibility.

OutputGeoJsonValidator

FeatureCollection, ровно один summary на variant_id, поля обязательных типов, cost/length totals, score formula, 2D/3D coordinate dimension.

API run status

scenario READY; run 10 TWO_D DONE; run 11 DEPTH DONE; progress 100; error null; validation_errors = [] для всех 6 вариантов.

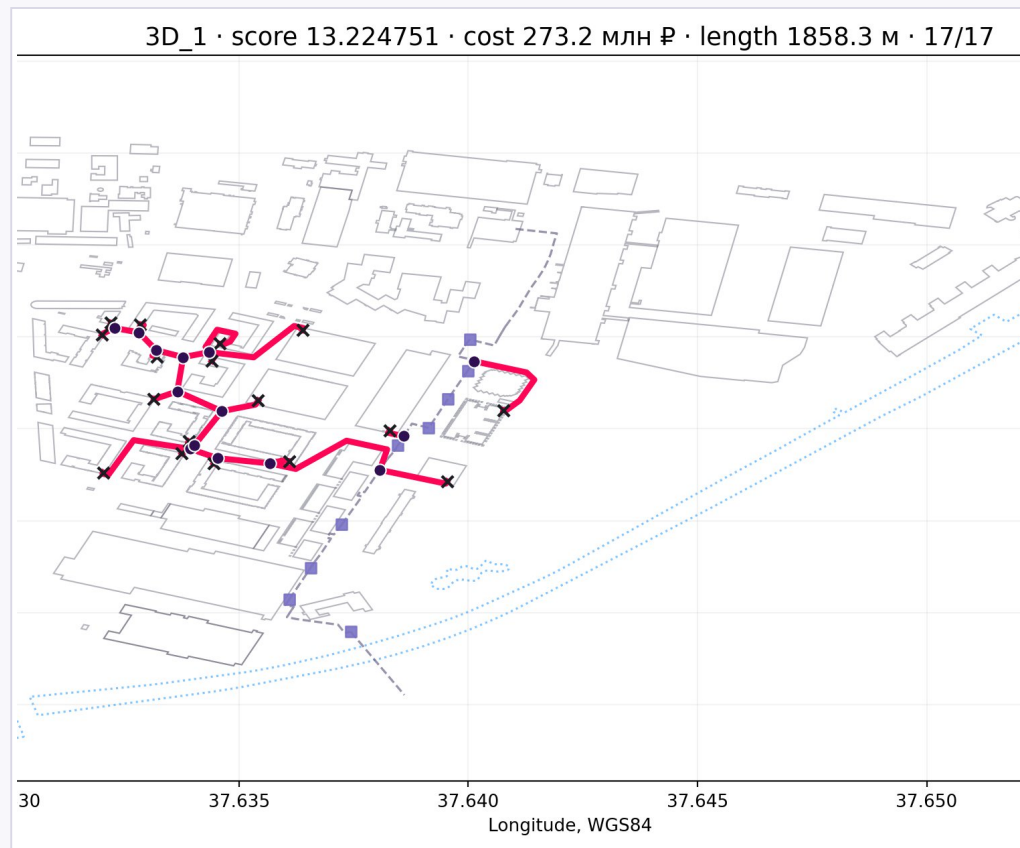
PASS

6/6 variants validation_passed=true

Ошибки и предупреждения валидатора отсутствуют. Это сильный аргумент для защиты: score не считается по «красивой картинке», а подтверждается независимым слоем.

Depth routing: отдельный режим, а не пост-обработка 2D

3D-выгрузка содержит координаты [lon, lat, -depth] и поля depth_start/depth_end



Модель глубины

Обычная глубина 3 м; минимум 0.7 м; при пересечении коммуникаций строятся рампы, плато и возврат к базовой глубине. Максимальный уклон — 0.10.

Стоимость глубины

$K_{\text{depth}} = 1 + 0.10 \times (h - 3)$ для глубины > 3 м; на подъёмах/спусках применяется средний коэффициент концов участка.

Что показывает текущий прогон

Depth-mode рассчитан и валидирован. На данном датасете лучшая topology совпала с 2D и не получила дополнительной стоимости: это нормальный результат, если выбранный коридор не требует заглубления сверх базовых правил.

В презентации не нужно утверждать, что 3D всегда улучшает score: корректная формулировка — режим есть, валидируется и не ухудшил финальный best-run.

MilpPortfolioSelector: выдаём не один ответ, а набор альтернатив

Эксперт получает до трёх содержательно различных вариантов, а не три сдвига одной линии

VariantDiversity

Сравниваются точки врезки, topology/grouping и corridor distance. Несовместимые пары исключаются.

Exact selection

Перебираются совместимые singleton/pair/triple; сначала максимизируется число вариантов, затем лучший individual score и сумма score.

Практический смысл

Вариант №1 лучше по score; №2 дешевле по CAPEX, но длиннее; №3 даёт резервный corridor/topology для инженерного сравнения.

variant	rank	policy	score	CAPEX	L
2d_1	1	INFRASTRUCTURE_LIGHT	13.2248	273.2	1858
2d_2	2	OFFICIAL_BEST	13.2321	268.8	1902
2d_3	3	OFFICIAL_BEST	13.3241	276.3	1863

Что пробовали и почему финальная архитектура именно такая

Решение не появилось как один скрипт: мы отсеяли подходы, которые хуже проходят инженерные ограничения

Подход	Плюс	Почему не финал
Independent shortest path	быстрый baseline	не оптимизирует общие магистрали, камеры и DN
Adaptive grid / raster A*	простая реализация обхода	лестничные маршруты, высокая цена сетки, сложнее фасадный вход
Greedy nearest-first	быстро подключает все ОКС	локальные решения ухудшают общую стоимость и topology
ALNS / beam в Python	хорошо исследует topology	тяжело переносить как production-core; оставлен как R&D-ориентир
Full external MILP	теоретически мощно	слишком большой search space + точная JTS-геометрия
ML/surrogate ranking	ускоряет candidate scoring	не гарантирует выполнение ТЗ; validator всё равно обязателен

Финальная комбинация

Sparse visibility graph + shared network forest + leaf reconnect + JointChamberMilp + full engineering validation. Это даёт баланс: достаточно сильный topology search, точная геометрия и воспроизводимый Java-production pipeline.

Техническая архитектура кода

Компоненты разделены по слоям: API, ingest, routing, optimization, engineering, validation, export

Слой	Ключевые классы	Ответственность
API	ScenarioController, RunController	загрузка сценария, запуск 2D/3D, статусы, variants, export
Ingest	GeoJsonStreamingIngestor, CrsTransformer	стриминг GeoJSON, EPSG:4326→32637, persistence
Routing	SparseVisibilityRouter, RouteConstraintEngine	DN-aware visibility graph, building entry, restrictions
Optimization	UniversalOptimizer, SharedNetworkBuilder, JointPlanRefiner	лес подключений, topology reconnect, совместное движение камер
Engineering	PlanEvaluator, FlowDiameterCalculator, RuleBook	flow aggregation, DN, chambers, official score
Validation/Export	EngineeringValidator, OutputGeoJsonValidator, GeoJsonResultWriter	PASS/FAIL, totals, output schema, streaming GeoJSON

Соответствие ТЗ: ключевые пункты закрыты end-to-end

Требования проверены не только описанием, но и фактическим локальным запуском

Требование	Статус	Аргумент
Java 11 / Spring Boot 2.6.3 / Docker Compose	DONE	pom.xml, Dockerfile, docker-compose, RUN_TASK
Потоковый input до 3 ГБ / output до 500 МБ	DONE	InputStream + JsonParser; export JsonGenerator
Все перспективные ОКС за один запуск	DONE	17/17 connected во всех 6 вариантах
Автоматический выбор врезок и совместное подключение	DONE	TieCandidateGenerator + SharedNetworkBuilder
Ограничения и дома	DONE	RouteConstraintEngine + EngineeringValidator
Расходы, DN, стоимость, score	DONE	RuleBook/PlanEvaluator; score_recalc delta = 0
До трёх вариантов и объяснимость	DONE	MilpPortfolioSelector + VariantDiversity
Depth-mode / 3D GeoJSON	DONE	DEPTH run DONE; coordinates include Z = -depth

Воспроизводимость и время расчёта

Длительный RUN_TASK — ожидаемое поведение, а не признак зависания

Почему может считать долго

UniversalOptimizer делает минимум 6 попыток / seed, при необходимости до 8; внутри каждого seed: build → junction refine → joint MILP → archive → reconnect leaves → local search → joint refine.

Что смотреть в логах

CANDIDATE mode=TWO_D seed=... score=... valid=true. Если эти строки появляются, расчёт идёт. RUN_TASK по умолчанию ждёт до двух часов на режим.

Как воспроизвести результат

Использовать тот же dataset, код и application.yml random-seed: 20260922. Выходные files: scenario.json, run-*.json, status-*.json, variants-*.json, result-*.geojson.

Как проверять руками

Открыть routes/*.geojson в QGIS/geojson.io, сверить PNG-карты, проверить variants-*.json: validation_passed=true, connected=17, errors/warnings пустые.

Сильные стороны решения для защиты

Это уже не «хакатонный маршрут», а техническое ядро инженерного сервиса

- Network-level optimisation: совместные магистрали, камеры, ветвления и topology families.
- Constraint-aware geometry: DN-aware buffers, фасадный вход в ОКС, запрет транзита через дома.
- Production-grade I/O: streaming upload/export, DB persistence, Docker, Swagger/API.
- Engineering validation: flow/DN/cost/score пересчитываются и проверяются независимо.
- Explainable variants: эксперт видит не только лучший вариант, но и альтернативы с другим балансом CAPEX/L.

Как запустить и показать жюри

Одна команда формирует тот же набор файлов, который использован в презентации

Основной запуск

```
./RUN_TASK.sh "task/Датасет скорректированный.geojson" results-final
```

Проверка статуса

```
cat results-final/status-2d.json
cat results-final/status-3d.json
cat results-final/variants-2d.json | jq
```

Что показать глазами

- 1) result-2d.geojson / result-3d.geojson в QGIS
- 2) routes/2d_1.geojson как лучший вариант
- 3) PNG-карты всех шести маршрутов
- 4) variants-*.json: validation_passed=true

Swagger / API

```
http://localhost:8080/swagger-ui.html
POST /api/v1/scenarios
POST /api/v1/scenarios/{id}/runs?mode=2d
GET /api/v1/runs/{id}/export
```

Лог прогресса

```
docker compose logs -f app
Искать строки CANDIDATE mode=... seed=... score=... valid=true
```

Backup: проверка score и totals

Числа в presentation напрямую восстановимы из GeoJSON summary и variants JSON

variant	C / 25M	L / 100	0.7C	0.3L	S
2d_1	10.9286	18.5825	7.6500	5.5748	13.224751
2d_2	10.7506	19.0223	7.5255	5.7067	13.232146
2d_3	11.0504	18.6295	7.7353	5.5888	13.324118

Наблюдение

Rank определяется не только CAPEX: у 2d_2 стоимость ниже, но длина выше, поэтому итоговый S хуже, чем у 2d_1.

Проверка

В metrics.csv для всех шести вариантов score_recalc совпадает с score; score_delta = 0.0.

Бэкап: тестовая сетка и контроль качества

Проект содержит unit/integration проверки для ключевых инженерных слоёв

- SparseVisibilityRouterTest — маршрутизация и ограничения
- RouteConstraintEngineTest — legal edge / препятствия
- FlowDiameterCalculatorTest — flow aggregation и DN
- JointChamberMilpTest — дискретный выбор камер
- MilpPortfolioSelectorTest — отбор трёх вариантов
- OutputGeoJsonValidatorTest — totals, schema, score
- DepthProfileTest — Z/depth, уклон, плато, просвет
- GeneralizationSmokeTest — отсутствие dataset-specific hardcode

ТЕПЛОКОНТУР

От GeoJSON — к проверенной инженерной конфигурации тепловой сети

17/17



ОКС подключены

13.2248



validated score

273.2 млн



итоговая стоимость

1.86 км



линейные работы

Network optimisation · Streaming I/O · Engineering validation · Java production architecture

Команда: [ФИО и роли участников] · Контакты: [email / telegram]